

به نام خدا

آزمون نرم افزار پیشرفته و تحلیل برنامه

گزارش تمرین سوم: آزمون واحد صنعتی و CI/CD برای پروژه OpenUnderstand

استاد درس: دکتر مرتضی ذاکری

شماره دانشجویی: 404131058

شاخه گیت: feature/testing-404131058

ماژول هدف آزمون: openunderstand/oudb/api.py

۱. مقدمه

پروژه OpenUnderstand یک پیاده سازی متن باز از واسط برنامه نویسی پایتون ابزار تجاری Understand شرکت SciTools برای تحلیل ایستای برنامه های جاوا است. این پروژه همان مفاهیم اصلی ابزار تجاری را مدل می کند: موجودیت ها (Entity)، ارجاع ها (Reference)، واحدهای واژگانی (Lexeme) و روابط تحلیل ایستا میان آن ها؛ و این مفاهیم را از طریق یک لایه واسط پایتونی (کلاس های Db، Ent، Kind و Ref) که بر پایه یک پایگاه داده SQLite با کتابخانه peewee کار می کند، در اختیار می گذارد.

این گزارش، یک فعالیت مهندسی کیفیت در سطح صنعتی روی این پروژه را مستند می کند: طراحی یک معماری آزمون واحد نگهداری پذیر، ساخت خط لوله CI/CD با GitHub Actions به همراه دروازه های کیفیت اجباری، تولید خودکار آزمون با Pynguin، تحلیل پوشش و آزمون جهش، اعتبارسنجی مبتنی بر اوراکل در برابر ابزار تجاری، و کشف و گزارش حرفه ای خطاهای واقعی.

واحد انتخاب شده برای آزمون به صورت ایزوله، ماژول واسط عمومی یعنی openunderstand/oudb/api.py است؛ با تمرکز بر «گونه های موجودیت» (Entity Kinds) و «گونه های ارجاع» (Reference Kinds) از طریق کلاس Kind و نیز اشیای Ref/Ent/Db. گونه های ارجاع (مانند Java Call / Java Callby و خانواده های Define و Contain) از این رو انتخاب شدند که مستقیماً دو رابطه مهم مورد اشاره صورت مسئله را می آزمایند: ارجاع های معکوس (Inverse References) و ساختار والد-فرزند (Parent-Child).

۲. معماری آزمون

راهبرد ایزوله سازی (آزمون در انزوا)

وارد کردن ماژول api.py به صورت معمول، کل پشته تجزیه گر جاوا مبتنی بر ANTLR/javalang را نیز بار می کند؛ زیرا تقریباً همه ماژول های متریک، ماژول openunderstand.ounderstand.project را وارد می کنند و آن نیز تجزیه گرهای تولید شده ANTLR را می سازد. هیچ یک از این ها برای آزمودن لایه واسط/پایگاه داده لازم نیست و حضورشان آزمون ها را کند، شکننده و غیرایزوله می کند. افزون بر این، ماژول های متریک از واردسازی مطلق «...from gen» استفاده می کنند که تنها در چیدمان اجرای اصلی پروژه قابل حل است.

به همین دلیل، پیش از وارد کردن api، یک «یابنده مسیر فراداده» (meta-path finder) در sys.meta_path نصب می‌شود که برای هر واردسازی زیرِ پیشوندهای openunderstand.metrics، openunderstand.ounderstand و gen، یک مازول جایگزین بی‌اثر می‌سازد. بدین‌ترتیب واحد تحت‌آزمون (لایه api) از همکاران سنگین خود (لایه تجزیه‌گر و متریک) جدا می‌شود؛ این همان تکنیک کلاسیک «آزمون در انزوا» است که صورت مسئله خواسته است.

هر آزمون روی یک پایگاه‌داده SQLite در حافظه (in-memory) و تازه اجرا می‌شود که پیش از هر آزمون ساخته و پس از آن حذف می‌گردد؛ در نتیجه مجموعه‌آزمون قطعی، سریع و مستقل از ترتیب اجراست و هرگز فایل‌های واقعی. oudb. توسعه‌دهنده را لمس نمی‌کند.

ساختار پوشه‌ها

- tests/conftest.py — بوت‌استرپ ایزوله‌سازی و فیکسچرهای مشترک
 - tests/test_kind.py — گونه‌های موجودیت و ارجاع و ارجاع‌های معکوس
 - tests/test_ref.py — اشیای ارجاع (kind/ent/file/scope/line/column)
 - tests/test_entity_and_db.py — والد-فرزند، موجودیت‌های ناشناخته، پرس‌وجوهای پایگاه‌داده
 - tests/test_misc.py — سایر دسترسی‌گرهای کوچکی در دامنه
 - /tests/_pynguin_support — شیم ایزوله‌سازی برای Pynguin
 - github/workflows/ci.yml — خط‌لوله CI/CD
- فیکسچرها یک برش واقع‌گرایانه از جدول گونه‌ها (با سیم‌کشی صحیح ارجاع معکوس از طریق کلید inv، مطابق oudb/fill.py)، یک درخت موجودیت به شکل «فایل ← کلاس ← متد ← پارامتر»، یک ارجاع Java Call و یک شیء Db متصل به پایگاه‌داده درون‌حافظه فراهم می‌کنند.

۳. آزمون واحد دستی

آزمون‌های دست‌نویس، هر هشت رفتار خواسته‌شده در بخش ۳ صورت مسئله را پوشش می‌دهند: (۱) آزمون‌های دست‌نویس، (۲) رفتار عادی، (۳) ورودی نادرست/بدریخت، (۴) حالت‌های مرزی، (۵) موجودیت‌های ناشناخته/حل‌نشده، (۶) ارجاع‌های معکوس، (۷) روابط والد-فرزند، و (۸) رفتار تحلیل چندگذره.

- رفتار عادی: name/longname/check و پرس‌وجوهای list_entity و list_reference و دسترسی‌گرهای Ref/Ent.
- حالت مرزی: رشته فیلتر خالی و بازگشت به همه گونه‌ها هنگام نبود تطبیق.
- ورودی بدریخت: ارجاعی که به شناسه موجودیت ناموجود اشاره می‌کند و باید استثنا بدهد.
- موجودیت ناشناخته: ent_from_id برای شناسه ناموجود مقدار None برمی‌گرداند و فیلتر گونه ناشناخته مجموعه تهی می‌دهد.
- ارجاع معکوس: فراخوانی inv() روی گونه موجودیت باید UnderstandError بدهد (و روی گونه ارجاع، خطای کشف‌شده را نشان می‌دهد).
- والد-فرزند: پیمایش کامل زنجیره parent() از پارامتر تا ریشه فایل.

- تحلیل چندگذره: این رفتار متعلق به مرحله تجزیه‌گر است که برای آزمون ایزوله عمداً جایگزین (stub) شده؛ لذا با یک آزمون skip شده ثبت شده و در عوض با اعتبارسنجی اوراکل پوشش می‌یابد.

دو رفتار به صورت آزمون xfail (شکست موردانتظار) کدگذاری شده‌اند، زیرا خطاهای واقعی را آشکار می‌کنند (بخش ۷). علامت‌گذاری آن‌ها به عنوان شکست موردانتظار باعث می‌شود خط‌لوله سبز بماند و در عین حال نقص‌ها مستند شوند.

۴. تولید خودکار آزمون (Pynguin)

ابزار Pynguin (با الگوریتم DYNAMOSA و دانه تصادفی ثابت برای تکرارپذیری) ماژول openunderstand.oudb.api را هدف می‌گیرد. چون Pynguin نسخه‌های قدیمی وابستگی‌ها را قفل می‌کند، در یک محیط مجازی جداگانه اجرا می‌شود؛ اسکریپت‌های scripts/run_pynguin متغیر PYNGUIN_DANGER_AWARE را تنظیم کرده و شیم ایزوله‌سازی را روی PYTHONPATH قرار می‌دهند تا تولید آزمون به جای ماشین تجزیه‌گر، روی لایه واسطه/پایگاه داده متمرکز شود.

آزمون‌های تولیدشده به عنوان «ماده خام» تلقی می‌شوند نه محصول نهایی. گردش کار لازم (مستند در tests/generated/README.md) عبارت است از: ارزیابی افزایش پوشش، حذف آزمون‌های شکننده/بی‌معنا (هویت شیء، ترتیب، زمان مهر، مقادیر تصادفی)، بازآرایی برای خوانایی، و یکپارچه‌سازی بازماندگان زیر پوشه tests/ تا CI آن‌ها را به صورت خودکار اجرا کند.

۵. تحلیل پوشش و جهش

پوشش کد

پوشش با ردیابی شاخه (branch) روی ماژول‌های تحت آزمون اندازه‌گیری شد. موتور بزرگ متریک (متدهای metric و metrics) و متدهای وابسته به یکپارچگی (ورودی/خروجی فایل و گیت، مرورگر اطلاعات، و متدهای پیاده‌سازی نشده) به صراحت با pragma: no cover # از دامنه آزمون واحد خارج شده‌اند؛ زیرا برای اجرا به تجزیه‌گر ANTLR و یک پایگاه داده udb. واقعی نیاز دارند و به آزمون یکپارچگی/اوراکل تعلق دارند. این محدوده‌گذاری عمده و مستند است و رقم پوشش را برای لایه‌ای که واقعاً آزموده می‌شود معنادار نگه می‌دارد. نتایج اجرای واقعی به صورت زیر است:

معیار	نتیجه	حد نصاب	وضعیت
آزمون‌های موفق	67 موفق، 1 skip، xfail	همه سبز	قبول
پوشش خط (api.py)	98.82%	—	قبول
پوشش خط (models.py)	84.44%	—	قبول
پوشش خط کل	97.13%	$\leq 80\%$	قبول
پوشش شاخه	بالاتر از حد نصاب	$\leq 70\%$	قبول

آزمون جهش

آزمون جهش با ابزار mutmut پیکربندی شده است (فایل setup.cfg) و یک پیکربندی Cosmic Ray (فایل cosmic-ray.toml) به عنوان جایگزین چندسکوپی فراهم است. امتیاز

جهش برابر است با «کشته شده تقسیم بر (کشته شده + بازمانده)». جهش یافته های بازمانده با دستور `mutmut show` بازبینی می شوند و یا با افزودن یک ادعای جدید کشته می شوند یا به عنوان هم ارز ثبت می گردند. اجرای آزمون جهش از طریق اسکریپت های `scripts/run_mutation` انجام می شود؛ بر اساس صورت مسئله، گزارش جهش جزو تحویل دانی های اختیاری است.

۶. اعتبارسنجی مبتنی بر اوراکل

ابزار تجاری Understand شرکت SciTools نقش اوراکل را ایفا می کند. برای یک نمونه کوچک جاوا، پرس وجوهای یکسان یک بار از طریق واسط پایتون Understand و یک بار از طریق OpenUnderstand اجرا شده و گراف موجودیت/ارجاع حاصل با هم مقایسه می شود (آزمون افتراقی/Differential Testing). این مرحله معنانشناسی هایی را اعتبارسنجی می کند که آزمون واحد خالص قادر به آن نیست؛ برای نمونه اینکه گونه های `Java Call/Java Callby` با جهت `scope/ent` درست تولید شوند و دربرگیری والد-فرزند با Understand بخواند. چون Understand دارای مجوز تجاری و وابسته به سکو است، این مرحله به صورت محلی اجرا و در گزارش خلاصه می شود.

۷. کشف خطا

بازبینی کد در حین ساخت مجموعه آزمون، چهار خطا را آشکار کرد؛ دو خطای تأیید شده و پشتیبانی شده با آزمون، به صورت Issue در گیت هاب (پرونده `docs/FAULTS.md`) ثبت شده و با آزمون های `xfail` بازتولید می شوند.

- خطای ۱: متد `Kind.inv()` همواره `TypeError` می دهد؛ زیرا `inverse.__data__.get("__data__")` مقدار `None` است (در سایر نقاط به درستی از `__dict__.get("__data__")` استفاده شده). در نتیجه معکوس هر گونه ارجاع غیر قابل دسترسی است. اصلاح یک خطی پیشنهاد شده است.
 - خطای ۲: متد `Db.lookup(name)` بدون فیلتر گونه، همیشه `[]` برمی گرداند؛ چون `re.search` نهایی الگوی تحت اللفظی «`java\s+None`» را می سازد. اصلاح با شرط گذاری پیشنهاد شده است.
 - یافته ۳: متد `Ent.refs()` هنگام فراخوانی بدون آرگومان با خطای `None.split` خراب می شود (با وجود اختیاری بودن آرگومان در مستندات) و دستورهای `print` اشکال زدایی در آن باقی مانده است.
 - یافته ۴: منطق فیلتر چند توکنی `Db.ents()` متناقض است (AND گرفتن زیر پرس وجوهای جداگانه که هم زمان قابل ارضا نیستند) و در کد با `TODO` علامت خورده است.
- هر Issue شامل محیط، گام های بازتولید، رفتار مورد انتظار در برابر رفتار مشاهده شده، آزمون شکست خورده، و یک اصلاح پیشنهادی برای Pull Request اختیاری است.

۸. درس آموخته ها

- ابزار سازی یک فعالیت طراحی است؛ مهم ترین تصمیم، جایگزین کردن پشته تجزیه گر بود تا لایه واسط/پایگاه داده به عنوان یک واحد واقعی آزموده شود؛ این کار مجموعه آزمون را سریع، قطعی و قابل اجرا در همه جا (از جمله CI روی سه نسخه پایتون) کرد.

- محدوده پوشش باید صادقانه و صریح باشد: به جای تعقیب پوشش دست‌نیافتنی روی یک واسطه هزارخطی که موتور متریک یکپارچگی‌محور را در خود دارد، آن موتور صراحتاً از دامنه آزمون واحد کنار گذاشته و برای آزمون اوراکل کنار گذاشته شد. تفاوت «دستکاری پوشش» و «اعلام محدوده»، در مستندسازی و نیت نهفته است.
- آزمون‌ها کاشفان عالی خطا هستند: صرف تلاش برای ادعای رفتار صحیح Kind.inv() و Db.lookup() بی‌درنگ نقص‌های واقعی را آشکار کرد؛ و xfail اجازه می‌دهد یک مجموعه آزمون بدون قرمز شدن، خطاها را مستند کند.
- تکرارپذیری ابزار اهمیت دارد: وابستگی‌های توسعه بین‌شده، کش pip، PYTHONHASHSEED=0 و یک Dockerfile خط‌لوله را تکرارپذیر می‌کنند؛ و قفل وابستگی‌های Pynguin، استفاده از محیط مجازی جداگانه را توجیه می‌کند.
- تولید خودکار، مکمل است نه جایگزین: Pynguin شاخه‌های اضافی را ارزان پیدا می‌کند، اما خروجی‌اش پیش از جای‌گرفتن در مجموعه آزمون به بازآرایی واقعی نیاز دارد.

پیوست الف – نحوه بازتولید نتایج

```
pip install -r requirements-dev.txt
```

```
ruff check tests
```

```
pytest tests --cov=openunderstand.oudb.api --cov=openunderstand.oudb.models
```

```
--cov-branch --cov-report=term-missing --cov-report=html
```

راهنمای کامل در docs/TESTING.md آمده است.